

Introduction aux circuits logiques programmables

Dr. Ghislain Kouadio (Enseignant-chercheur)

12 novembre 2025

UE : Électronique numérique
Niveau : Licence 1 : BD, DAS, RSI, MMX
Volume : CM 2h + TD 2h
Établissement : *Université Virtuelle de Côte d'Ivoire*

Résumé de la leçon

Cette leçon introduit les familles de circuits logiques programmables (PROM, PLA, PAL, CPLD, FPGA), leurs architectures et le flux de conception HDL (VHDL/Verilog), avec un accent sur les FPGA modernes et leurs applications (traitement du signal, télécoms, systèmes embarqués).

Prérequis

Logique combinatoire et séquentielle de base, algèbre de Boole, additionneur/régistre ; notions de systèmes numériques.

Objectifs d'apprentissage

À l'issue, l'étudiant(e) sera capable de :

- distinguer PROM/PLA/PAL/CPLD/FPGA et situer historiquement leur apparition ;
- décrire l'architecture des circuits logiques programmables ;
- décrire le cas particulier de l'architecture d'un FPGA ;
- connaître les principes de base de la programmation des circuits logiques programmables ;
- décrire le flux de conception HDL (VHDL / Verilog).

1. Introduction et mise en contexte historique

Depuis l'avènement de l'électronique numérique, les ingénieurs ont cherché à concevoir des systèmes logiques de plus en plus complexes et performants. Dans un premier temps, ces systèmes étaient construits à l'aide de circuits câblés fixes, composés de portes logiques élémentaires (ET, OU, NON, etc.) connectées de manière définitive. Si cette approche convient aux conceptions simples, elle atteint rapidement ses limites dès que la complexité du circuit augmente : manque de flexibilité, consommation d'espace importante, difficulté de mise à jour ou de correction d'erreurs.

Pour surmonter ces contraintes, une nouvelle génération de composants a vu le jour : les circuits logiques programmables (Programmable Logic Devices – PLD). Contrairement aux circuits logiques câblés classiques, ces dispositifs permettent de définir ou de reconfigurer la logique interne après fabrication, en fonction des besoins spécifiques de l'utilisateur. Ils offrent ainsi un compromis idéal entre la souplesse du logiciel et la rapidité du matériel. Aujourd'hui, les circuits logiques programmables occupent une place centrale dans de nombreux domaines : traitement du signal et de l'image, télécommunications, électronique embarquée, ou encore prototypage rapide de processeurs. Parmi eux, les FPGA (Field Programmable Gate Array) constituent l'une des technologies les plus répandues et les plus puissantes, permettant d'implémenter des systèmes complexes allant d'un simple additionneur à un processeur complet.

Les circuits logiques programmables ont émergé pour dépasser les limites des circuits câblés fixes. Des PROM (années 1970) aux FPGA (depuis 1985), l'évolution a suivi la croissance des besoins en flexibilité et intégration.

- **PROM** : programmation fusible, tables de vérité simples.
- **PLA/PAL** : matrices AND/OR programmables (ou semi-fixes) → plus de variété logique.
- **CPLD** : agrégation de macrocells + interconnexion globale.
- **FPGA** : grille de CLB (LUT+FF), réseau d'interconnexions, I/O, BRAM, DSP, parfois CPU (SoC).

Cette leçon vise donc à explorer les différents types de circuits logiques programmables, leur architecture, leur fonctionnement, ainsi que les méthodes de conception et de programmation associées.

2. Présentation des circuits logiques programmables

2.1 Présentation de la logique programmée

Il y a quelques années la réalisation d'un montage en électronique numérique impliquait l'utilisation d'un nombre important de circuits intégrés logiques. Ceci avait pour conséquences, un prix de revient élevé, une mise en œuvre complexe et un circuit imprimé de taille importante.

Le développement des mémoires utilisées en informatique fut à l'origine des premiers circuits logiques programmables (**PLD : Programmable Logic Device**). Les circuits logiques programmables sont utilisés pour remplacer l'association de plusieurs boîtiers logique. Le câblage est simplifié, l'encombrement et le risque de pannes est réduit. Certains PLD ne permettent pas la relecture de la fonction logique programmée, c'est pratique lorsque le programme doit rester confidentiel.

Ces circuits disposent d'un certain nombre de broches d'entrées et de sorties. L'utilisateur associe ces broches aux équations logiques (plus ou moins complexes) qu'il programme dans le circuit.

Sa mise en œuvre se fait très facilement à l'aide d'un programmeur, d'un micro-ordinateur et d'un logiciel adapté.

Pour des moyennes séries, les composants programmables sont en général préférés aux ASIC. Les PLD peuvent être programmés par l'utilisateur (bureau d'étude, chaîne d'assemblage) alors que les ASIC qui sont configurés lors de leur fabrication (ils ne peuvent être que simulés par le bureau d'étude) sont réservés à des productions de très grande série en raison de leur coût de développement élevé.

Le désavantage majeur est qu'il faut se procurer le matériel nécessaire pour programmer les modules programmables. Puisque chaque compagnie fabrique des composants programmables différents, le changement de composants, de compagnies ou de technologies peut être coûteux. Ces désavantages affectent plus particulièrement le développement de nouveaux produits et ces coûts deviennent insignifiants lors de la production à grande échelle.

2.2 Présentation d'un PLD

Les PLD (circuits logiques programmables) sont des circuits vierges disposant des entrées et des sorties dont l'utilisateur peut programmer le schéma logique d'après les besoins liés à la fonction logique souhaitée (cahier des charges) : logique combinatoire et/ou logique séquentielle.

2.3 Structure d'un PLD

La plupart des PLDs suivent la structure de base de la figure 1. En effet, Un PLD comprend généralement trois parties principales :

- Les entrées et inverseurs
 - Les signaux d'entrée $E_1, E_2, \dots, E_n$ sont chacun disponibles sous deux formes :
 - * la forme directe E_i ,
 - * la forme complémentée $\bar{E}_i$.
 - Chaque entrée passe donc par un inverseur, ce qui permet d'utiliser soit la variable, soit son complément dans les équations logiques.
- Le plan ET programmable (Matrice ET)
 - Ce plan réalise les produits logiques (AND) entre les différentes entrées ou leurs inverses.
 - Chaque croisement entre une ligne d'entrée et une colonne du plan ET peut être connecté (programmé) ou déconnecté, selon la fonction à implémenter.
 - Ce plan permet donc de générer tous les mintermes nécessaires.
- Le plan OU programmable (Matrice OU)
 - Le plan OU combine les produits générés par le plan ET pour former les sommes logiques correspondant aux sorties du circuit.
 - Chaque sortie S_i est une combinaison (somme logique) de plusieurs produits programmés :
- Les sorties : chaque sortie peut correspondre à une fonction logique combinatoire ou, dans les PLD plus évolués, à une fonction séquentielle (avec bascules D, par exemple).

L'ensemble des opérateurs forment des matrices (matrice OU et matrice ET). Ces deux matrices forment une structure appelée PLA (Programmable Logic Array) ou un réseau logique programmable. Les interconnexions de ces matrices sont assurées par des fusibles qui sont « grillés » lors de la programmation. Lorsqu'un PLD est vierge toutes les connexions sont assurées (fusibles intacts).

Une matrice programmable est un ensemble de portes logiques dont les entrées sont connectées aux variables du système, vraies et inversées, au moyen d'un réseau de fusibles. Lorsqu'un fusible est brûlé, la connexion respective entre la variable et la porte logique disparaît. La programmation du système se fait en choisissant les fusibles que l'on laisse et que l'on brûle.

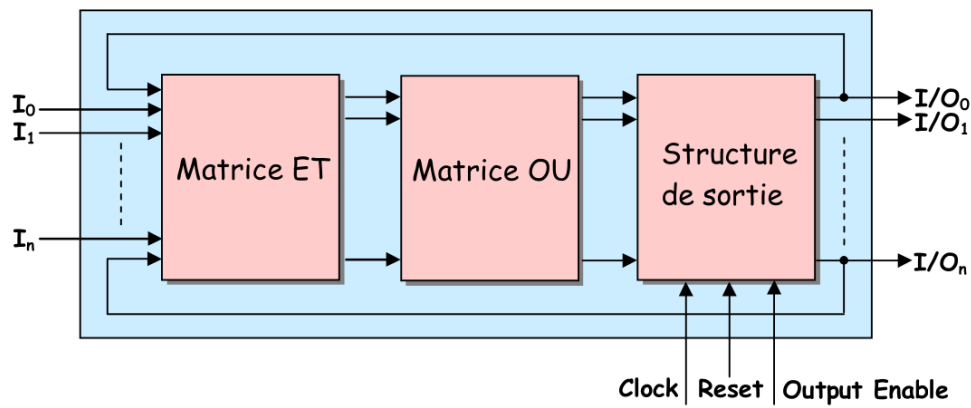


FIG. 1 : Structure de base d'un circuit logique programmable (PLD).

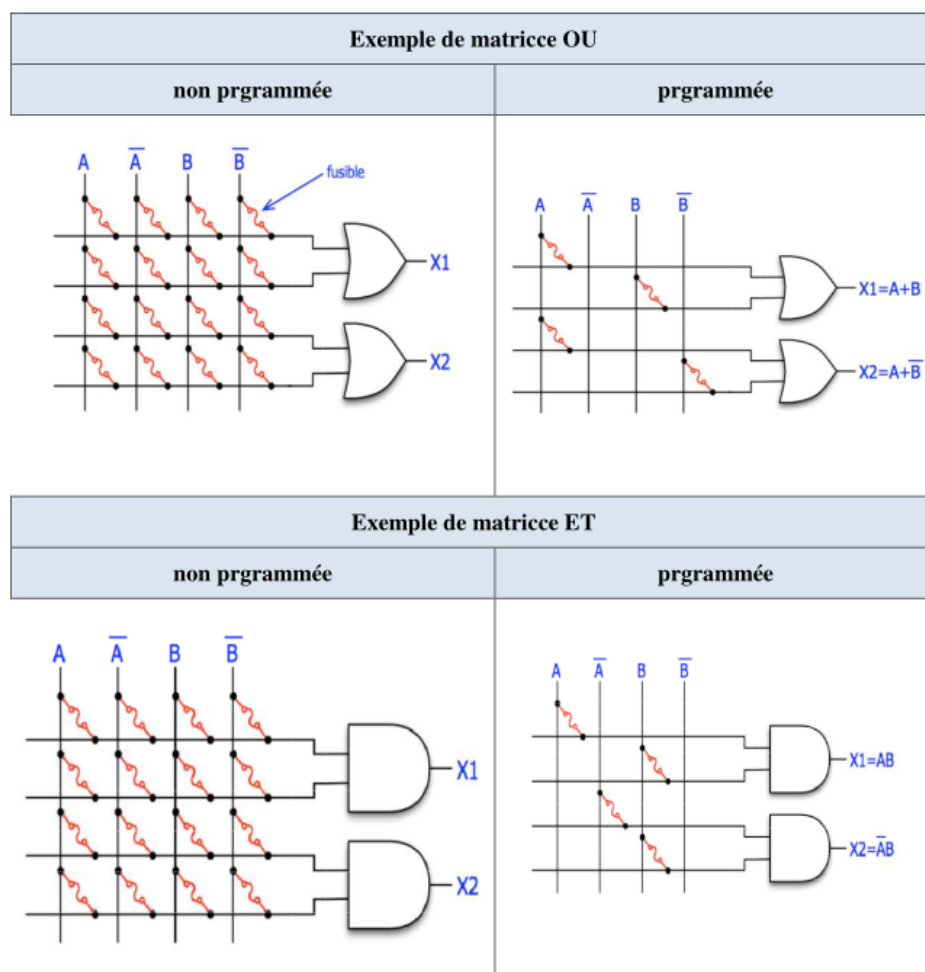


FIG. 2 : Exemples de matrices programmables (OU et ET) en configuration non programmée et programmée.

Remarque

- $/x$ signifie $\bar{x}$ (voir figure 1).

- La technique des fusibles est en voie de disparition : le PLD est programmé une seule fois.
- Actuellement la technique la plus répandue utilise le transistor MOSFET à la place du fusible. Cette technique permet de programmer et reprogrammer le PLD grâce à un effacement électrique.

2.4 Principe général de fonctionnement

1. Les entrées et leurs inverses sont disponibles.
2. Le plan ET programmable sélectionne et combine ces entrées pour créer les produits logiques souhaités.
3. Le plan OU programmable additionne ces produits pour générer les fonctions de sortie.
4. Les sorties peuvent être directement combinatoires ou reliées à des bascules pour des circuits séquentiels.

Selon le caractère programmable des matrices ET et OU, il existe trois types de circuits logiques programmables :

- **PROM** (**P**rogrammable **R**ead-**O**nly **M**emory) : la matrice ET est fixe et la matrice OU est programmable. C'est une mémoire.
- **PLA** (**P**rogrammable **A**rray **L**ogic) : matrice ET programmable suivie d'une matrice OU fixe.
- **PLA** (**P**rogrammable **L**ogic **A**rray) : les deux matrices sont programmables.

Les circuits PALs sont les PLDs les plus courants : les PROMs sont utilisées en tant que mémoires et les PLAs sont très peu utilisés à cause de leur nombre élevé de fusibles, ce qui amène une grande consommation de puissance et de long délai.

2.5 Représentation et symbolisation

La figure ci-dessous représente la structure simplifiée d'un PAL (2 entrées et 1 sortie).

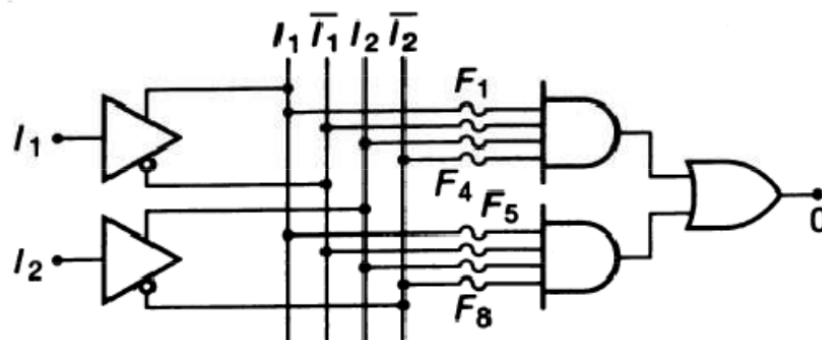
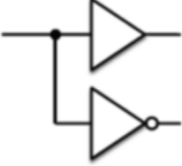
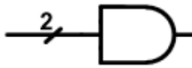
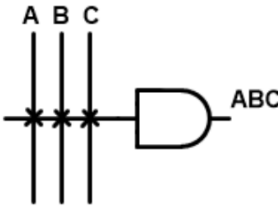


FIG. 3 : Structure simplifiée d'un PAL.

Cette représentation schématique demande beaucoup d'espace pour représenter un PAL en entier. Les industriels ont adopté la représentation symbolique suivante :

Circuit normal	Circuit équivalent
	
	
	

3. Classification des réseaux logiques combinatoires

Selon le caractère programmable des matrices ET et OU, il existe trois types de circuits logiques programmables :

1. PROM (Programmable Read-Only Memory) : la matrice ET est fixe et la matrice OU est programmable. C'est une mémoire : la matrice ET sert de décodeur d'adresse ; pour chaque valeur d'adresse, la PROM produit une valeur qui a été programmée. Les PROMs sont utilisées en tant que mémoires.
2. PAL (Programmable Array Logic) : matrice ET programmable suivie d'une matrice OU fixe. Les circuits PALs sont les PLDs les plus courants. Les PAL sont les circuits logiques programmables les plus anciens. Les PAL sont programmés par destruction de fusibles. Ils ne sont donc programmables qu'une fois, ce qui peut être gênant en phase de développement.
3. PLA (Programmable Logic Array) : les deux matrices sont programmables. Les PLAs sont très peu utilisées à cause de leur nombre élevé de fusibles, ce qui amène une grande consommation de puissance et de longs délais.

4. Familles de circuits logiques programmables

Il existe plusieurs familles de PLD qui sont différenciées par leur structure interne. La diversité de constructeurs de circuits logiques programmables est à l'origine de plusieurs produits équivalents mais d'appellation différente. En effet, il est difficile d'établir une classification générale de ces circuits. La figure ci-dessous présente l'une des classifications adoptées. Le tableau et le digramme suivants présentent certaines de ces familles.

Type	Nombre de portes intégrées	Matrice ET	Matrice OU	Effaçable
PROM	2 000 à 500 000	Fixe	Programmable	Non
PAL	10 à 100	Programmable	Fixe	Non
GAL	10 à 100	Programmable	Fixe	Électriquement
EPLD	100 à 3 000	Programmable	Fixe	Aux U-V
FPGA	2 000 à 3 000	Programmable	Programmable	Électriquement

TAB. 1 : Classification des familles de circuits logiques programmables.

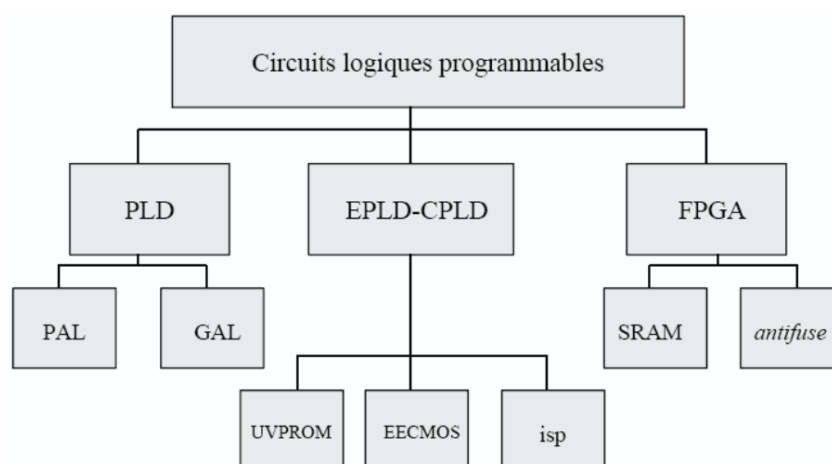


FIG. 4 : Classification des familles de circuits logiques programmables.

4.1 PAL

PAL signifie Programmable Array Logic, c'est à dire réseau logique programmable. La programmation de ces circuits s'effectue par destruction de fusibles. Une fois programmés on ne peut plus les effacer. On distingue deux sous familles :

- Les PAL combinatoires ou PAL simples. Ils sont constitués de fonctions de logique combinatoire.
- Les PAL à registres ou FPLS (Field Programmable Logic Sequencer) pour séquenceur logique programmable. Ils sont constitués de logique combinatoire et séquentielle (Registre).

4.2 EPLD (PAL effaçable)

EPLD signifie Erasable Programmable Logic Device, c'est à dire circuit logique programmable et effaçable et qui sont aux PAL ce que sont les UVROM. aux PROM. Les EPLD peuvent être effacés par ultraviolet (UV) ou électriquement. Ils sont encore appelés PAL CMOS.

4.3 GAL

GAL signifie Generic Array Logic ou encore réseau logique générique ce qui veut dire pas grand chose mais qui sont aux PAL ce que sont les EEPROM aux PROM. Le nom de GAL a été déposé par LATTICE SEMICONDUCTOR. Leur fonctionnement est identique aux PAL CMOS, ils sont programmables et effaçables électriquement.

4.4 CPLD

CPLD signifie Complex Programmable Logic Device. Ces circuits sont composés de plusieurs PALs élémentaires (Par exemple l'équivalent de PALs 22V10) reliés entre-eux par une zone d'interconnexion. Grâce à cette architecture, ils permettent d'atteindre des vitesses de fonctionnement élevées (plusieurs centaines de Mhz).

4.5 LCA et FPGA à antifusible

LCA signifie Logic Cell Array ou encore réseau de cellules logiques. Ces circuits sont composés de blocs logiques élémentaires de 2000 à 10000 portes que l'utilisateur peut interconnecter.

FPGA signifie Forecasting Programmable Gate Array. Les FPGA à anti-fusibles sont identiques aux LCA sauf qu'ils permettent une plus grande intégration de portes et ils ne sont pas effaçables électriquement. Le nom anti-fusible vient de la programmation des connexions qui s'effectue par fermeture de circuits, comparé aux fusibles où l'on ouvre les circuits.

5. Architectures et principe de fonctionnement des PLDs

Toute équation logique à n variables peut se mettre sous la forme d'une somme de produits ou de produits de somme.

Tout naturellement les composants programmables disposent d'une structure comportant ces deux ensembles fonctionnels :

- Un ensemble d'opérateurs ET (matrice ET) recevant les entrées et leurs compléments.

- Un ensemble d'opérateurs OU (matrice OU) qui réalise une somme de produits et fournit les sorties.

Selon la complexité des circuits programmables utilisés, l'une ou l'autre ou chacune des matrices pourra être programmée.

Lorsqu'elles sont programmables, les matrices utilisent des "fusibles" placés en série entre les entrées et les opérateurs logiques.

5.1 PROM et PAL

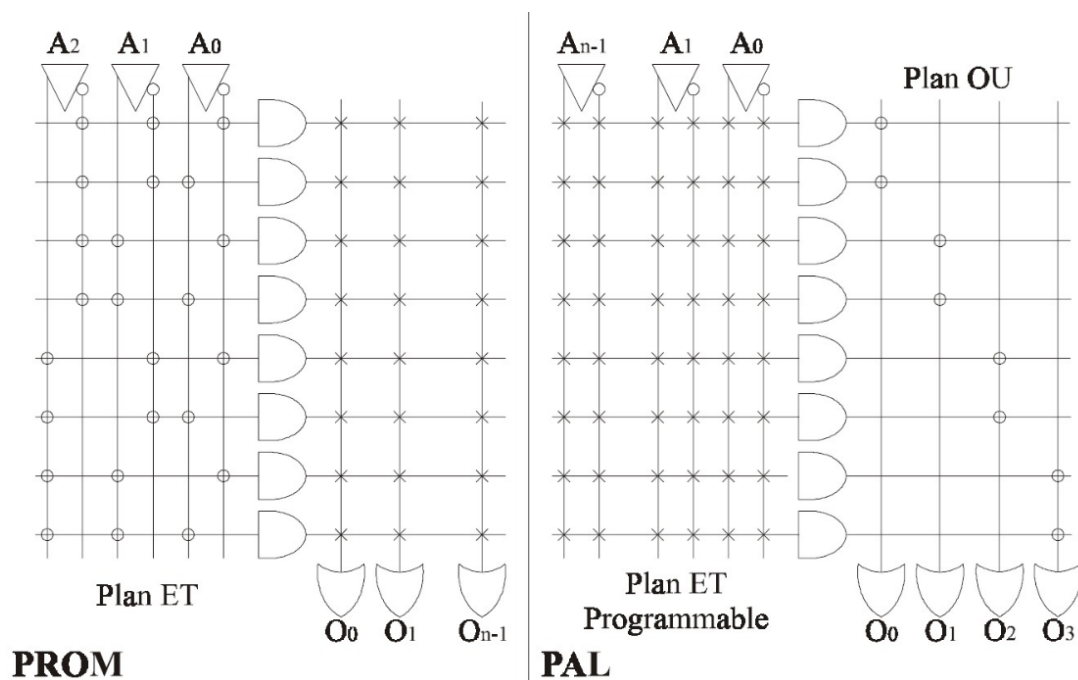


FIG. 5 : Architectures d'un PROM et d'un PAL.

5.1.1 Structure

- Entrées logiques : correspondent aux adresses binaires.
- Plan ET fixe : décode toutes les combinaisons possibles des entrées (2^n lignes).
- Plan OU programmable : permet de relier certaines lignes à des sorties via des fusibles programmables.
- Sorties : représentent les valeurs logiques définies par la table de vérité.

5.1.2 Principe de fonctionnement

1. Codage de la fonction logique :

- Chaque combinaison des entrées correspond à une adresse mémoire.
- Chaque bit stocké dans la PROM définit la valeur de sortie pour cette combinaison.

2. Programmation :

- Les connexions (fusibles) du plan OU sont détruites ou laissées intactes pour définir la logique souhaitée.
- Une fois programmée, la PROM ne peut plus être modifiée.

3. Lecture / utilisation :

- Les entrées sélectionnent directement la ligne correspondant à la combinaison des variables.
- La sortie est le bit stocké à cette adresse.

5.2 PLA

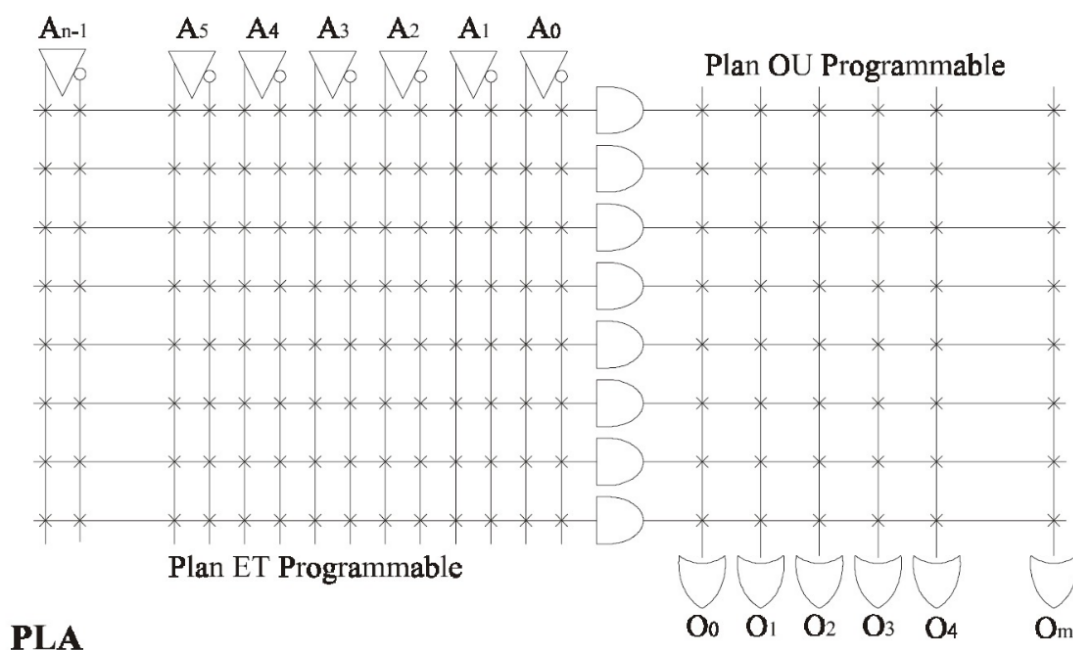


FIG. 6 : Architecture d'un PLA.

5.2.1 Structure

1. Entrées et inverseurs : chaque entrée ($A, B, \dots$) est disponible sous deux formes : directe et complémentée.
2. Plan ET programmable :
 - Les croisements entre lignes d'entrée et lignes de produit logique ET sont des fusibles programmables.
 - Chaque produit logique (ou terme implicant) peut être choisi librement.
3. Plan OU fixe :
 - Les produits issus du plan ET sont combinés dans des portes OU non programmables.

- Chaque porte OU réalise une équation somme de produits (forme canonique).
4. Sorties : parfois reliées à des bascules D ou JK (pour logique séquentielle simple).

5.2.2 Principe de fonctionnement

1. Programmation :
 - On choisit quels croisements (dans le plan ET) doivent être actifs : cela détermine les termes produits nécessaires.
 - Le plan OU fixe combine ces produits pour générer les fonctions de sortie.
2. Utilisation : lorsqu'un ensemble d'entrées satisfait un produit logique programmé, la sortie correspondante est activée.

6. Architecture interne d'un FPGA

6.1 Définition d'un FPGA

Un FPGA (Forecasting Programmable Gate Array) est un circuit logique reconfigurable après fabrication, c'est-à-dire que sa structure interne peut être programmée pour réaliser pratiquement n'importe quelle fonction logique ou système numérique.

Le FPGA est un réseau (matrice) de blocs combinatoires et séquentiels (CLB). Il se compose principalement de :

- Blocs d'entrée/sortie (IOB) sont associés aux broches du circuit.
- Les CLB et IOB sont interconnectés entre eux par des dispositifs variés. Ces réseaux d'interconnexions sont programmables.
- Ressources spéciales (mémoire, horloges, DSP, etc.).

Les FPGAs apportent une grande facilité d'emploi et permettent une réduction du coût de développement des électroniques embarquées. Les FPGAs reconfigurables à base de SRAM (Static Random Access Memory) apportent en outre la possibilité de faire évoluer in situ la fonctionnalité de ces circuits, ce qui ouvre des perspectives de maintenance à distance que n'autorisent pas les circuits spécifiques.

6.2 Les différents types de FPGA

Le type de FPGA dépend de la façon dont la configuration interne (CLB, interconnexions, I/O) est mémorisée et reprogrammée.

6.2.1 FPGA SRAM (Static RAM-based)

- Principe : la configuration est stockée dans des cellules SRAM volatiles.

- Caractéristiques :
 - Reprogrammable à volonté (nombre illimité de fois).
 - Configuration perdue à la mise hors tension → nécessite un fichier de configuration (bitstream) chargé à chaque démarrage (souvent depuis une mémoire Flash externe).
- Avantages :
 - Très flexible, rapide, facile à reprogrammer.
 - Prise en charge de configurations partielles dynamiques.
- Inconvénients :
 - Nécessite un support mémoire externe.
 - Sensible aux perturbations électromagnétiques (volatilité).
- Fabricants typiques :
 - Xilinx (AMD) : séries Spartan, Artix, Kintex, Virtex.
 - Intel (ex-Altera) : séries Cyclone, Arria, Stratix.

6.2.2 FPGA Flash

- Principe : la configuration est stockée dans des cellules Flash non volatiles.
- Caractéristiques :
 - La configuration est conservée même sans alimentation.
 - Reprogrammable plusieurs milliers de fois.
- Avantages :
 - Démarrage instantané (pas besoin de mémoire externe).
 - Plus résistants au rayonnement et à la corruption mémoire.
 - Consommation plus faible que les FPGA SRAM.
- Inconvénients :
 - Temps de reprogrammation plus long.
 - Densité logique et performance légèrement inférieures aux SRAM.
- Fabricants : Microchip (ex-Microsemi / Actel) : séries ProASIC, IGLOO, SmartFusion.

6.2.3 FPGA Antifuse

- Principe : la configuration est obtenue par création physique de connexions permanentes (les antifuses passent d'un état ouvert à fermé).
- Caractéristiques :
 - Programmation unique (One-Time Programmable, OTP).
 - Connexions permanentes, non modifiables.
- Avantages :
 - Excellente fiabilité, sécurité et stabilité (non reconfigurable).

- Très faible consommation et haute performance.
- Immunisé contre les radiations (idéal pour l'aérospatial, la défense).
- Inconvénients : Impossible de reprogrammer après fabrication.
- Fabricants : Microchip (ex-Actel) : séries Axcelerator, SX-A, RTAX (version rad-hard).

Les matrices s'organisent de 8×8 à 128×120 .

6.3 Blocs logiques (CLB) et LUT

Les CLB (Configurable Logic Blocks) constituent le cœur du FPGA.

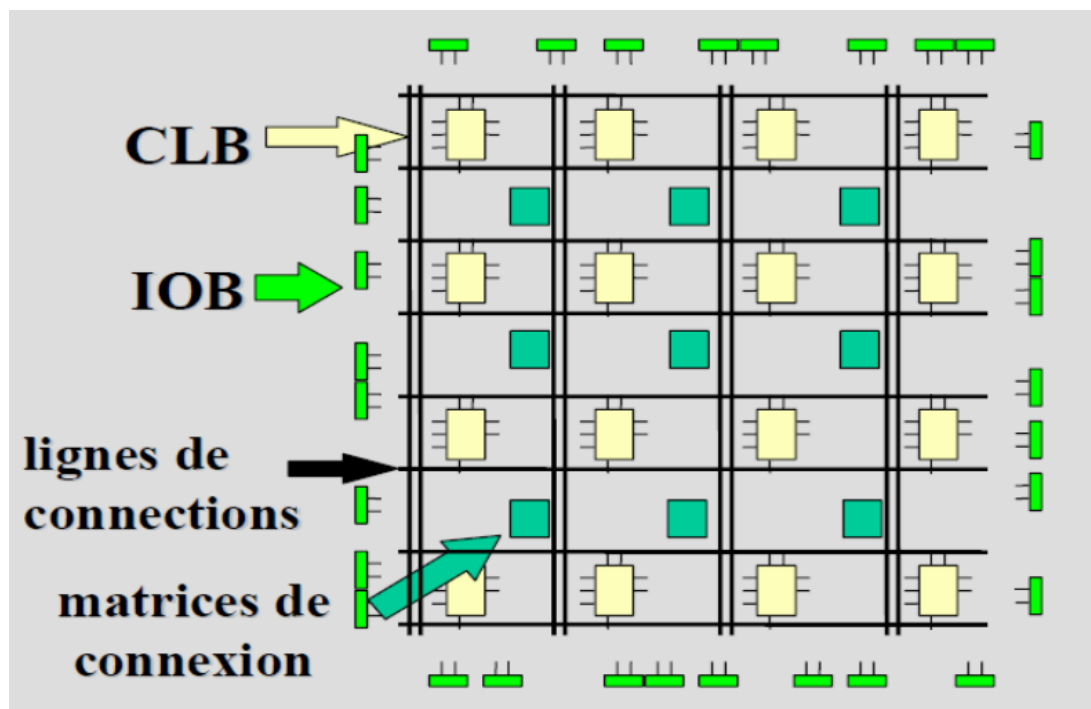


FIG. 7 : Architecture générale d'un FPGA.

6.3.1 Structure interne

Chaque CLB est formé de :

- LUT (Look-Up Table) : petite mémoire (souvent 4 à 6 entrées) capable de réaliser n'importe quelle fonction logique combinatoire.
 - Une LUT à 4 entrées peut implémenter n'importe quelle fonction de 4 variables (16 possibilités).
 - Elle agit comme une table de vérité programmable.
- Flip-flops (FF ou registres) : utilisés pour le stockage des états logiques, la synchronisation et la mise en œuvre de circuits séquentiels.

- Multiplexeurs et multiplexeurs locaux : sélectionnent les signaux internes ou externes à transmettre à la sortie.
- Chaînes de transport (carry chains) : optimisent les additions/soustractions en accélérant la propagation des retenues.

6.3.2 Fonctionnement

1. La LUT implémente la logique combinatoire.
2. Le flip-flop enregistre le résultat si l'on souhaite une logique séquentielle.
3. Les multiplexeurs permettent de choisir entre signaux combinatoires ou séquentiels à la sortie du CLB.

6.4 Réseau d'interconnexions

Le réseaux d'interconnexion relie les CLB entre eux, ainsi qu'aux blocs d'E/S et aux ressources spéciales. Il existe trois type d'interconnexions entre les différents blocs des circuits FPGA.

- Interconnexion directe : entre les différents blocs logiques.
- Interconnexions par le biais d'une matrice.
- Interconnexion par les grandes lignes relient tous les CLB dans les extrémités des circuits FPGA.

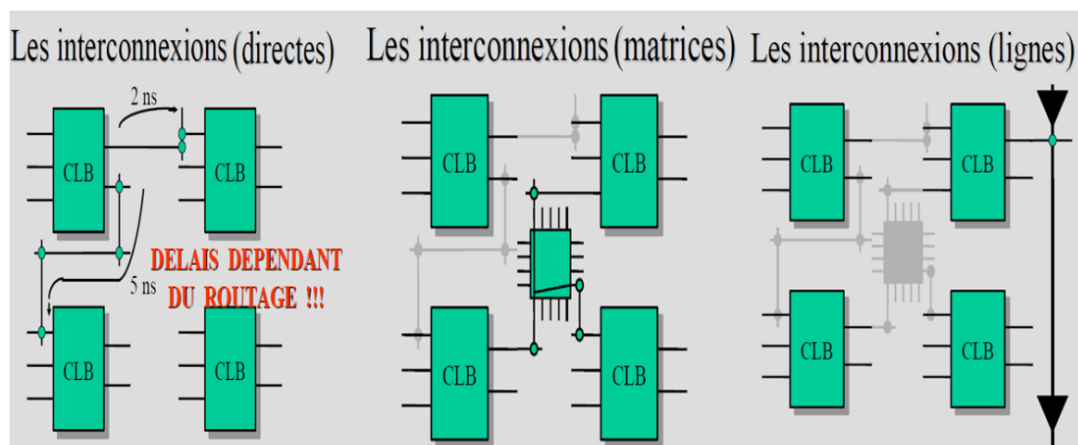


FIG. 8 : Les différentes méthodes d'interconnexions entres blocs logiques.

6.4.1 Composants

Ce réseaux d'interconnexions comprend :

- des canaux horizontaux et verticaux : pistes métalliques transportant les signaux.
- des commutateurs programmables (switch matrices) : points de connexion commandés (souvent par SRAM ou antifuse).

- Segments de longueur variable : pour relier des blocs proches (segments courts) ou éloignés (segments longs).

6.4.2 Objectif

Ce réseaux assure la flexibilité du routage avec :

- Connexion des signaux logiques entre les LUT.
- Acheminement des horloges, signaux de contrôle et données.
- Optimisation automatique lors de la synthèse et du placement-routage.

6.5 Blocs d'Entrée/Sortie (I/O Blocks – IOB)

Les IOB forment l'interface entre le FPGA et le monde extérieur.

6.5.1 Fonctions principales

- Adaptation de tension et de standards : LVTTTL, LVCMOS, HSTL, SSTL, etc.
- Buffer d'entrée/sortie : protège et conditionne les signaux.
- Registres d'entrée et de sortie : synchronisation avec l'horloge interne.
- Programmation directionnelle : une même broche peut être configurée en entrée, sortie ou bidirectionnelle.

6.5.2 Types de signaux gérés

- Signaux numériques rapides
- Lignes de communication série (SPI, UART, I²C...)
- Interfaces DDR (Double Data Rate)

6.6 Ressources spéciales

Les FPGA modernes intègrent plusieurs blocs dédiés pour améliorer les performances :

6.6.1 Blocs de mémoire

- BRAM (Block RAM) : mémoires intégrées (souvent 18 à 36 kbits) accessibles en simple ou double port.
- Distributed RAM : réalisée à partir des LUT.
- FIFO, ROM configurables.

6.6.2 Blocs DSP (Digital Signal Processing)

Les Blocs DSP contiennent des multiplicateurs–additionneurs matériels rapides et sont utilisés pour le traitement du signal, le filtrage, la FFT, etc.

6.6.3 Réseau d'horloge (Clock Network)

- PLL / DCM : circuits de gestion et de multiplication/division de fréquence.
- Global clock tree : distribue l'horloge avec un minimum de skew.

6.6.4 Blocs spécialisés

- Transceivers haut débit (SERDES)
- Interfaces Ethernet, PCIe, HDMI, etc.

7. Programmation des circuits logiques programmables (PLD)

7.1 Introduction

Les PLDs (Programmable Logic Devices) sont des circuits logiques numériques dont la fonction peut être configurée par l'utilisateur après fabrication. La programmation consiste à établir ou supprimer des connexions logiques à l'intérieur du circuit pour obtenir le comportement désiré. Selon le type de PLD (PROM, PAL, GAL, CPLD, FPGA), la méthode de programmation et la technologie physique varient.

7.2 Principe général

Le principe consiste à :

- Décrire la fonction logique (en équations booléennes, en table de vérité ou en langage matériel HDL : VHDL/Verilog).
- Traduire cette description en un ensemble de connexions à établir ou à couper dans la matrice logique du PLD.
- Charger la configuration dans le composant au moyen d'un programmeur ou d'un câble de développement (selon la technologie).

7.3 Étapes de la programmation

1. **Conception logique** : définition de la fonction souhaitée sous forme d'équations logiques ou de table de vérité ou d'un fichier HDL (VHDL, Verilog).
2. **Compilation et simulation** : le logiciel de développement (ex. Quartus, Vivado, WinCUPL) analyse la description logique, vérifie la syntaxe et simule le comportement attendu.
3. **Placement et routage** : le compilateur assigne les produits logiques aux cellules internes (LUT ou macrocells) et les interconnexions entre blocs logiques.
4. **Génération du fichier de configuration** : le logiciel crée un fichier binaire contenant les données de configuration (exemples : .jed (JEDEC), .pof, .sof, .bit, selon le type de

PLD).

5. **Chargement dans le composant** : le fichier est chargé dans le circuit via un programmeur externe (PROM, PAL, GAL) ou un câble JTAG / ISP (CPLD, FPGA modernes).

7.4 Mode de programmation

1. Programmation hors-circuit (off-line)

- Le composant est retiré du circuit et inséré dans un programmeur externe.
- Typique pour : PROM, PAL, GAL.
- Fichier de configuration → signaux électriques → fusibles ouverts/fermés.

2. Programmation en circuit (In-System Programming, ISP)

- La programmation s'effectue sans retirer le composant.
- Via interface JTAG ou port série SPI.
- Typique pour : CPLD, FPGA, GAL modernes.

3. Reconfiguration dynamique

- Dans certains FPGA SRAM, on peut modifier la configuration en fonctionnement.
- Permet l'adaptation en temps réel (systèmes reconfigurables).

8. Les langages de description matérielle (HDL)

8.1 Définition

Un HDL (Hardware Description Language) est un *langage de description matérielle* utilisé pour modéliser, simuler et synthétiser le comportement et la structure des circuits logiques numériques (portes, registres, bascules, processeurs, etc.).

Contrairement aux langages de programmation classiques (comme C, Python ou Java), les HDL ne décrivent pas des séquences d'instructions à exécuter, mais des connexions logiques et temporelles entre composants matériels fonctionnant en parallèle.

Un code HDL permet donc de représenter un circuit à différents niveaux d'abstraction :

- Niveau comportemental : description des fonctions logiques sans détail du matériel.
- Niveau RTL (Register Transfer Level) : description en termes de transferts entre registres synchronisés par l'horloge.
- Niveau structurel : interconnexion explicite des composants logiques.

8.2 But et intérêt

L'objectif principal d'un HDL est de :

- Décrire précisément le matériel sous forme textuelle.

- Simuler le comportement du circuit avant fabrication.
- Vérifier et valider la logique du design.
- Synthétiser le code en un circuit réel sur **FPGA** ou en **ASIC**.
- Réduire les erreurs de conception et les coûts de développement.

Ainsi, le HDL est un outil fondamental pour passer d'une idée logique à une implémentation matérielle réelle.

8.3 Les principaux langages HDL

Les deux langages HDL les plus utilisés sont :

VHDL (VHSIC Hardware Description Language)

Développé pour le Département de la Défense américain (DoD) dans les années 1980 pour documenter et simuler des circuits intégrés rapides (VHSIC). C'est un langage fortement typé, rigoureux, très utilisé dans l'enseignement, la recherche et les systèmes critiques (aéronautique, spatial).

Verilog HDL

Créé par Gateway Design Automation en 1984, standardisé par l'IEEE (1364). Syntaxe plus concise, proche du langage C, donc très répandue dans l'industrie électronique et le développement ASIC.

SystemVerilog

Extension moderne du Verilog (IEEE 1800), intégrant des fonctionnalités de **vérification orientée objet**, de modélisation à haut niveau et de conception de test (UVM).

Autres HDL spécialisés

D'autres langages plus spécifiques existent : *SystemC* (pour la co-simulation matériel/logiciel), *MyHDL* (basé sur Python), ou encore *Chisel* (langage HDL moderne basé sur Scala).

8.4 Synthèse

Les HDL constituent le lien entre la **conception logique** et la **réalisation physique** d'un circuit numérique. Ils permettent aux ingénieurs de concevoir, tester et optimiser un système avant toute fabrication matérielle.

Exemple VHDL : Porte OR à deux entrées

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity or2 is
5  port(a,b : in std_logic; y : out std_logic);
6  end entity;
7
8  architecture rtl of or2 is
9  begin
10     y <= a or b;
11 end architecture;
```

Listing 1: Description VHDL d'une porte OR à deux entrées

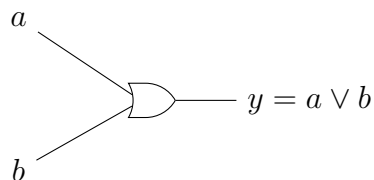
Analyse du code

Lignes 1–2 : Inclusion de la bibliothèque `ieee` et du paquet `std_logic_1164`.

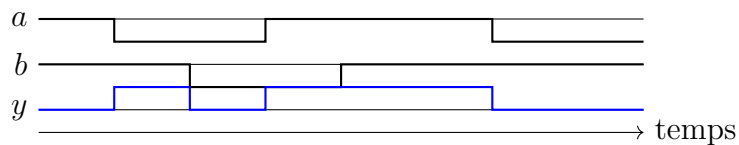
Lignes 3–5 : `entity` définit l'interface externe (`a`, `b` entrées; `y` sortie).

Lignes 6–9 : `architecture` décrit le comportement concurrent : `y <= a or b`;

Schéma logique



Chronogramme



Conclusion Ce code décrit une **porte logique OR à deux entrées**. L'entité `or2` définit les ports d'entrée et de sortie, tandis que l'architecture `rtl` exprime le comportement logique : la sortie `y` est égale à `a OR b`. Lorsqu'une des entrées passe à '1', la sortie devient '1'.

Exemple Verilog : Demi-additionneur (Half Adder)

```
1  module half_adder(input a, input b, output sum, output carry);
2
3  assign sum    = a ^ b;
4  assign carry = a & b;
5
6  endmodule
7
```

Listing 2: Demi-additionneur (Verilog)

Explication ligne par ligne

Ligne 1 `module half_adder(input a, input b, output sum, output carry);`
Déclaration du module nommé `half_adder`. Il expose deux entrées `a` et `b`, et deux sorties `sum` (somme) et `carry` (retenue). Les mots-clés `input/output` définissent la direction des ports.

Ligne 2 `assign sum = a ^ b;`

Affectation continue (`assign`) : la sortie `sum` est en permanence le XOR des entrées. L'opérateur `^` réalise la somme binaire sans retenue ($\text{sum} = a \oplus b$).

Ligne 3 `assign carry = a & b;`

Affectation continue : la sortie `carry` vaut le ET logique des entrées. L'opérateur `&` calcule la retenue ($\text{carry} = a \wedge b$).

Ligne 4 `endmodule`

Fin de la définition du module `half_adder`.

Rappel fonctionnel Un demi-additionneur additionne deux bits `a` et `b` :

$$\text{sum} = a \oplus b, \quad \text{carry} = a \wedge b.$$

9. Flux de conception HDL (VHDL/Verilog)

9.1 Définition

Le flux de conception HDL (ou HDL design flow) désigne l'ensemble des étapes de développement d'un circuit numérique à partir d'une description matérielle écrite en VHDL ou Verilog, jusqu'à sa programmation sur un FPGA ou sa fabrication en ASIC (Application-Specific Integrated Circuit, ou en français circuit intégré spécifique à une application).

Il s'agit d'un processus itératif : à chaque étape, des simulations ou vérifications permettent d'assurer la conformité du design aux spécifications.

Un ASIC est un circuit intégré conçu pour une tâche précise, contrairement aux circuits logiques programmables (comme les PLD, CPLD ou FPGA) qui peuvent être reprogrammés après fabrication. Il est donc fabriqué sur mesure pour exécuter une fonction particulière déterminée dès la phase de conception.

9.2 Étapes

Voici une description détaillée du flux de conception HDL (Hardware Description Language), applicable à VHDL comme à Verilog, qui décrit le processus complet de développement d'un circuit logique numérique, depuis la spécification jusqu'à la mise en œuvre sur une puce (FPGA ou ASIC).

9.2.1 Spécification du système

C'est la phase initiale où l'on définit ce que doit faire le circuit :

- Fonctions à réaliser (additionneur, contrôleur, processeur, etc.)
- Contraintes de performance (fréquence, latence, consommation, surface, coût, etc.)
- Interfaces d'entrée/sortie (signaux, protocoles, temporisations)

➤ **Livrable** : cahier des charges fonctionnel.

9.2.2 Conception comportementale (modélisation HDL)

Le concepteur décrit le comportement logique du circuit en langage HDL (VHDL ou Verilog) :

- Utilisation de structures comme process, always, if, case, etc.
- Modélisation du chemin de données et du contrôleur.
- Cette étape peut être faite au niveau RTL (Register Transfer Level), c'est-à-dire en termes de registres, d'horloges et de transferts de données.

➤ **Exemple** : description RTL d'un additionneur, d'un compteur, ou d'un multiplexeur.

9.2.3 Simulation comportementale

Avant toute synthèse, on simule le code HDL pour vérifier son bon fonctionnement logique :

- Vérification des séquences d'entrée/sortie avec un testbench (banc d'essai).
- Observation des signaux temporels dans un simulateur (ModelSim, QuestaSim, Vivado Simulator, etc.).
- Validation fonctionnelle (le code fait bien ce qu'on veut).

➤ **Livrable** : modèle validé au niveau fonctionnel.

9.2.4 Synthèse logique

Le code HDL est ensuite synthétisé par un outil (Synplify, Vivado, Quartus, Design Compiler, etc.) :

- Traduction du code HDL en réseau logique (portes, bascules, LUT, etc.)
- Optimisation selon les contraintes (temps, surface, puissance)
- Génération d'un netlist (fichier de description du circuit sous forme de composants logiques interconnectés).

➤ **Livrable** : fichier netlist (souvent au format EDIF ou BLIF).

9.2.5 Simulation post-synthèse

La simulation basée sur la netlist synthétisée :

- Vérifie que la logique est inchangée après optimisation.
- Permet d'estimer les temps de propagation et de vérifier la conformité temporelle.

➤ **Livrable** : validation logique et temporelle avant implantation.

9.2.6 Placement et routage (Implementation / Place and Route)

Pour un FPGA :

- L'outil d'implémentation (Vivado, Quartus, etc.) place chaque élément logique (LUT, FF, I/O) sur la puce.
- Il route ensuite les connexions physiques entre les blocs.

Pour un ASIC :

- Cette étape correspond au placement physique des cellules standard et au routage du silicium.
- On obtient alors le layout du circuit.

➤ **Livrable** : fichier de configuration (bitstream pour FPGA) ou masque (GDSII pour ASIC).

9.2.7 Simulation post-implémentation

- On vérifie que les délais réels du matériel respectent les contraintes d'horloge.
- Analyse temporelle statique (STA) pour garantir la tenue des chemins critiques.
- Simulation post-routage pour détecter d'éventuels problèmes de synchronisation.

9.2.8 Programmation et validation sur matériel

Pour un FPGA :

- Génération du fichier bitstream (.bit).

- Téléchargement sur la carte via JTAG ou USB.
- Tests en conditions réelles avec signaux d'entrée/sortie physiques.

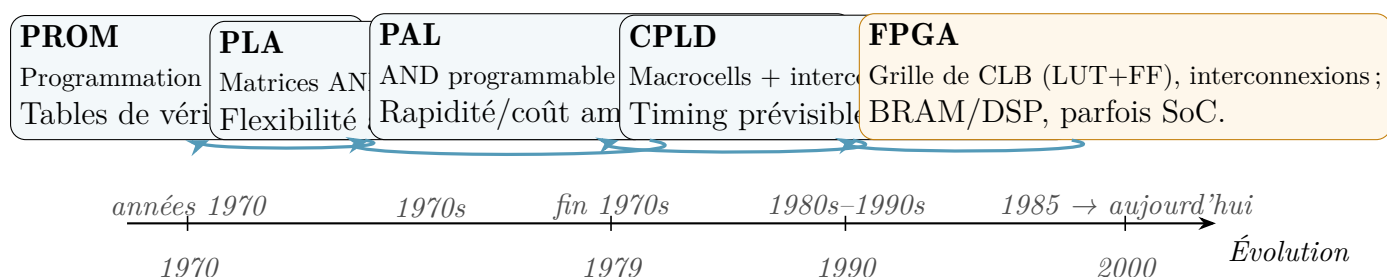
Pour un ASIC :

- Fabrication en fonderie.
 - Tests sur silicium (test de production, test de conformité).
- **Livable** : circuit matériel opérationnel.

9.2.9 Vérification et validation finales

- Comparaison entre les performances réelles et les spécifications initiales.
- Validation fonctionnelle, temporelle, et électrique.
- Documentation finale du design et archivage des fichiers source, netlist, scripts, et rapports.

Annexes



Dates indicatives : l'apparition commerciale et la diffusion académique peuvent varier selon les fabricants.

FIG. 9 : Ligne du temps des circuits logiques programmables : PROM → PLA → PAL → CPLD → FPGA.